توسط پردازه‌های دیگر جلوگیری کند. برای اشتراک حافظه لازم است که دو یا چند پردازه برای حذف این محدودیت با هم به توافق برسند. به این ترتیب، می‌توانند اطلاعات را با خواندن و نوشتن داده‌ها در این بخش‌های مشترک مبادله کنند. شکل و مکان داده‌ها توسط این پردازه‌ها تعیین می‌شود و تحت کنترل سیستم‌عامل نیست. این پردازه‌ها هستند که باید اطمینان بدهند همزمان یک مکان حافظه را نمی‌نویسند. این راهکارها در فصل ۶ بحث می‌شوند. در فصل ۴، گونه‌ای از طرح پردازه—*نخ‌ها*[1]—را خواهیم دید که مقداری از حافظه را به طور پیش‌فرض به اشتراک می‌گیرند.

هر دو مدلی که در اینجا بحث کردیم در سیستم‌عامل‌ها رایج هستند و بیشتر سیستم‌عامل‌ها هر دو را پیاده‌سازی می‌کنند. عبور پیغام برای تبادل حجم کوچک‌تر داده‌ها مفید است، چون نیازی به اجتناب از برخوردها ندارد. پیاده‌سازی آن نیز برای ارتباط میان کامپیوترها نسبت به حافظه‌ی مشترک ساده‌تر است. ارتباط حافظه‌ی مشترک، حداکثر سرعت را داشته و مناسب است، چرا که می‌تواند در یک کامپیوتر با سرعت‌های حافظه انجام بگیرد. با این وجود، مسائلی هم در حوزه‌های حفاظت و همگامی میان پردازه‌هایی که حافظه را به اشتراک می‌گیرند، وجود دارد.

۲-۳-۳-۶ حفاظت

حفاظت، راهکاری برای کنترل دسترسی منابعی است که توسط سیستم کامپیوتری فراهم می‌شوند. از نظر تاریخی، حفاظت فقط دغدغه‌ی سیستم‌های کامپیوتری چندبرنامگی با کاربران متعدد بود. با این وجود، با ظهور شبکه و اینترنت، تمامی سیستم‌های کامپیوتری از کارگزارها گرفته تا دستگاه‌های دستی موبایل بایستی نگران حفاظت باشند.

نوعاً، فراخوانی‌های سیستمی که حفاظت را فراهم می‌کنند عبارتند از set_permission() و get_permission()، که تنظیمات مجوز منابعی چون فایل‌ها و دیسک‌ها را دستکاری می‌نمایند. فراخوانی‌های allow_user() و deny_user() مشخص می‌کنند که آیا کاربران خاصی می‌توانند —یا نمی‌توانند— به منابع معینی دسترسی داشته باشند. حفاظت را در فصل ۱۷ و ملاحظات امنیتی بیشتر را—که شامل استفاده از حفاظت در مقابل تهدیدات خارجی است—در فصل ۱۶ پوشش می‌دهیم.

۲-۴ سرویس‌های سیستمی

وجه دیگر یک سیستم مدرن، مجموعه‌ی سرویس‌های سیستمی آن است. شکل ۱-۱ را به خاطر بیاورید که سلسله‌مراتب منطقی کامپیوتر را نشان می‌داد. سخت‌افزار در پایین‌ترین سطح قرار دارد. بعد از آن سیستم‌عامل، سپس سرویس‌های سیستمی و در نهایت، برنامه‌های کاربردی قرار می‌گیرند. **سرویس‌های سیستمی** که به **برنامه‌های سودمند سیستمی**[2] نیز معروفند، محیطی ساده و راحت را برای توسعه و اجرای برنامه فراهم می‌کنند. برخی از آنها به عنوان رابط‌های

1. Threads
2. System utilities

کاربر ساده‌ای برای فراخوانی‌های سیستمی هستند. برخی دیگر فوق‌العاده پیچیده هستند. می‌توان آنها را به دسته‌های زیر تقسیم کرد:

- **مدیریت فایل**: این برنامه‌ها برای ایجاد، حذف، کپی، تغییر نام، چاپ، لیست‌گیری و در حالت کلی، فایل‌ها و دایرکتوری‌ها را دسترسی و دستکاری می‌نمایند.
- **اطلاعات وضعیتی**: برخی برنامه‌ها به سادگی تاریخ، ساعت، مقدار حافظه یا فضای دیسک موجود، تعداد کاربران یا اطلاعات وضعیتی مشابه را از سیستم درخواست می‌کنند. برنامه‌های دیگر پیچیده‌ترند و اطلاعات با جزئیات بیشتری از کارایی، سابقه و عیب‌یابی را فراهم می‌کنند. این برنامه‌ها نوعاً خروجی را قالب‌بندی کرده و در ترمینال یا سایر دستگاه‌های خروجی یا فایل‌ها چاپ می‌کنند یا در پنجره‌ای از GUI نمایش می‌دهند. برخی سیستم‌ها یک **رجیستری** هم دارند که برای ذخیره و بازیابی اطلاعات پیکربندی استفاده می‌شود.
- **تغییر فایل**: ویرایشگرهای متنی متعددی ممکن است برای ایجاد و تغییر محتویات فایل‌های ذخیره شده روی دیسک یا سایر دستگاه‌های ذخیره‌سازی در دسترس باشند. برای جستجوی محتویات فایل‌ها یا تبدیل متن آنها نیز امکان دارد فرمان‌های خاصی هم وجود داشته باشند.
- **پشتیبانی زبان برنامه‌نویسی**: کامپایلرها، اسمبلرها، عیب‌یاب‌ها و مفسرهای زبان‌های برنامه‌نویسی (نظیر C، C++، جاوا، و پایتون) غالباً توسط سیستم‌عامل یا به صورت download جداگانه برای کاربر فراهم می‌شوند.
- **بارگذاری و اجرای برنامه**: همین که یک برنامه اسمبل یا کامپایل می‌شود، بایستی به منظور اجرا در داخل حافظه بار شود. سیستم ممکن است بارکنندگان مطلق[1]، بارکنندگان قابل تغییر مکان[2]، ویرایشگرهای پیونددهنده[3] و بارکنندگان جایگذاشت[4] را فراهم نماید. سیستم‌های خطایابی برای زبان‌های سطح بالا و یا زبان ماشین نیز لازم می‌شوند.
- **ارتباطات**: این برنامه‌ها راهکار ایجاد اتصال‌های مجازی مابین پردازه‌ها، کاربران و سیستم‌های کامپیوتری را مهیا می‌کنند. همین‌طور، امکان ارسال پیغام کاربران به صفحه‌نمایش کاربران دیگر، مرور صفحات وب، ارسال پیغام‌های پست‌الکترونیکی، ورود از دور به سیستم، یا انتقال فایل‌ها از یک ماشین به ماشین دیگر را فراهم می‌کنند.
- **سرویس‌های پس‌زمینه**: تمامی سیستم‌های همه‌منظوره متدهایی برای اجرای پردازه‌های خاص سیستم–برنامه در زمان بالاآمدن دارند. برخی از این پردازه‌ها با اتمام وظایف خود خاتمه می‌یابند، در حالی که بقیه تا توقف سیستم به اجرای خود ادامه می‌دهند. پردازه‌های سیستم–برنامه که مداوم اجرا دارند به **سرویس‌ها**، **زیرسیستم‌ها**، یا گماشته‌ها معروفند. یک مثال، گماشته‌ی شبکه است که در بخش

1. Absolute loaders
2. Relocatable loaders
3. Linkage editors
4. Overlay loaders

۲-۳-۳-۵ بحث می‌شود. در آنجا، سیستم نیاز به سرویسی دارد که برای اتصالات شبکه گوش دهد تا درخواست‌ها را به پردازه‌های درست برساند. مثال‌های دیگر عبارتند از: زمانبندهای پردازه که پردازه‌ها را بر اساس یک زمانبندی مشخص شروع می‌کنند، سرویس‌های ناظر خطای سیستم و کارگزارهای چاپ. سیستم‌های نوعی چندین جین گماشته دارند. به علاوه، سیستم‌عامل‌هایی که فعالیت‌های مهمی را در زمینه‌ی کاربر به جای زمینه‌ی هسته اجرا می‌کنند ممکن است برای اجرای این فعالیت‌ها از گماشته‌ها استفاده کنند.

اکثر سیستم‌عامل‌ها به همراه برنامه‌های سیستمی، برنامه‌هایی دارند که در حل مشکلات یا انجام عملیات رایج سودمند هستند. این قبیل **برنامه‌های کاربردی** مرورگرهای وب، واژه‌پردازها و قالب‌دهندگان متون، صفحه‌های گسترده، سیستم‌های پایگاه‌داده‌ها، کامپایلرها، بسته‌های نرم‌افزاری ترسیم و تحلیل آماری و بازی‌ها را در بر می‌گیرند.

دیدی که بیشتر کاربران از سیستم‌عامل دارند، به جای فراخوانی‌های سیستمی واقعی، توسط برنامه‌های کاربردی و برنامه‌های سیستمی تعریف می‌شود. کامپیوتر شخصی یک کاربر را در نظر بگیرید. زمانی که کامپیوتر کاربر، سیستم‌عامل macOS را اجرا می‌کند، کاربر امکان دارد رابط کاربر گرافیکی را ببیند که امکانات ماوس و پنجره‌ها را برای او فراهم می‌کند. در شرایطی دیگر، ممکن است کاربر در یکی از پنجره‌های خود، پوسته‌ای از خط فرمان یونیکس را هم داشته باشد. هر دو از مجموعه‌ی یکسانی از فراخوانی‌های سیستمی استفاده می‌کنند، اما فراخوانی‌های سیستمی به ظاهر متفاوتی دارند و به روش‌های متفاوتی عمل می‌کنند. حتی دید کاربر از این هم پیچیده‌تر می‌شود، کاربری را در نظر بگیرید که با بوت دوگانه از macOS به ویندوز وارد می‌شود. اکنون، همان کاربر بر روی سخت‌افزار یکسان، دو رابط و دو مجموعه برنامه‌ی کاربردی کاملاً متفاوتی دارد که از منابع فیزیکی یکسان استفاده می‌کنند. در اینجا، کاربر روی یک سخت‌افزار می‌تواند رابط‌های کاربر متعدد را به ترتیب یا همروند در اختیار داشته باشد.

۲-۵ پیونددهنده‌ها و بارگذارها

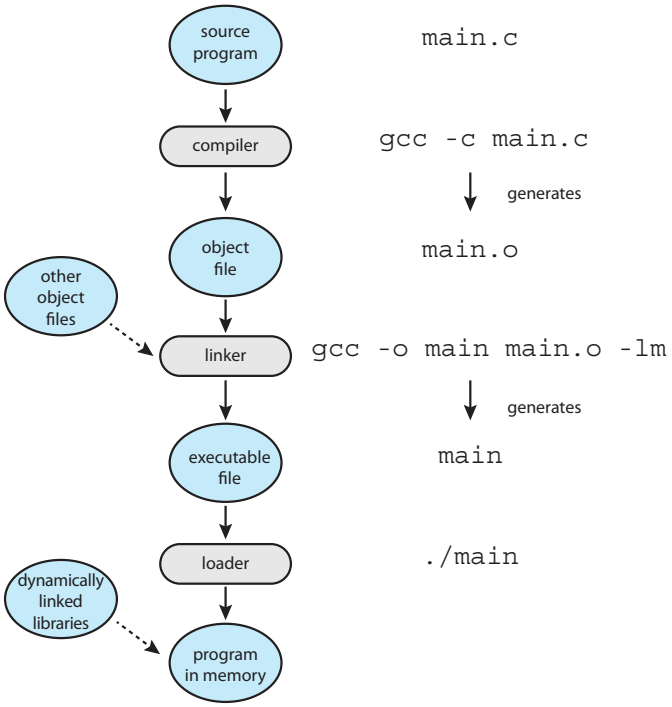
معمولاً، یک برنامه به‌صورت یک فایل اجرایی دودویی بر روی دیسک مقیم است—برای مثال، a.out یا prog.exe. برنامه به‌منظور اجرا بر روی یک CPU بایستی به داخل حافظه آورده شده و در قالب یک پردازه قرار بگیرد. در این بخش، گام‌های این روال را بیان می‌کنیم، از کامپایل یک برنامه گرفته تا قراردادن آن در داخل حافظه، جایی که برای اجرا بر روی یک هسته‌ی CPU در دسترس آمادگی پیدا می‌کند. این گام‌ها در شکل ۲-۱۱ به‌صورت برجسته نشان داده می‌شوند.

فایل‌های کد اصلی به‌شکل فایل‌های مقصد[1] کامپایل می‌شوند که طراحی شده‌اند تا در هر مکانی از حافظه بارگذاری شوند، قالبی تحت عنوان **فایل مقصد جابجاپذیر**[2]. در ادامه، **اتصال‌دهنده**[3] این فایل‌های مقصد جابجاپذیر را به‌شکل یک فایل **اجرایی** دودویی منفرد ترکیب می‌کند. در طی

1. Object files
2. Relocatable object file
3. Linker

مرحله‌ی اتصال، فایل‌های مقصد یا کتابخانه‌های دیگری ممکن است اضافه شوند، همچون کتابخانه‌ی استاندارد C یا math (که با پرچم lm– مشخص می‌شوند).

برای بارگذاری فایل اجرایی دودویی به‌داخل حافظه که در آنجا برای اجرا بر روی یک هسته‌ی CPU آمادگی پیدا می‌کند، از یک **بارگذار**[1] استفاده می‌شود. فعالیت مرتبط با اتصال و بارگذاری **تجدید مکان**[2] است که آدرس‌های نهایی را به بخش‌های یک برنامه نسبت می‌دهد و کد و داده‌های برنامه را تطبیق می‌دهد تا با آن آدرس‌ها تطابق پیدا کنند، برای مثال، کد می‌تواند همچنان‌که اجرا می‌شود، توابع کتابخانه‌ای را فراخوانی کرده و متغیرهای آن را دستیابی کند. در شکل ۲–۱۱ می‌بینیم که کل کاری که برای اجرای بارگذار لازم است واردکردن نام فایل اجرایی در خط فرمان است. زمانی که نام یک برنامه در خط فرمان سیستم‌های یونیکس وارد می‌شود— برای مثال، ./main—پوسته نخست یک پردازه‌ی جدیدی ایجاد می‌کند تا برنامه را با استفاده از فراخوانی سیستمی fork() اجرا کند. پوسته در ادامه، بارگذار را ضمن ارسال نام فایل اجرایی به‌عنوان آرگومان، فراخوانی سیستمی exec() را احضار می‌کند. سپس، بارگذار برنامه‌ی مشخص شده را با استفاده از فضای آدرس پردازه‌ای که به‌تازگی ایجاد شده به‌داخل حافظه بارگذاری می‌کند. (موقعی که یک رابط GUI استفاده می‌شود، کلیک مضاعف بر روی آیکن مربوط به فایل اجرایی، بارگذار را با استفاده از یک راهکار مشابه احضار می‌کند.)



شکل ۲–۱۱ نقش اتصال‌دهنده و بارگذار.

1. Loader
2. Relocation

فرایندی که تاکنون بیان شد فرض می‌کند که تمامی کتابخانه‌ها به فایل اجرایی متصل شده و داخل حافظه بارگذاری می‌شوند. در حقیقت، اکثر سیستم‌ها به یک برنامه اجازه می‌دهند تا کتابخانه‌ها را همچنان‌که برنامه بارگذاری می‌شود به‌صورت پویا متصل کنند. برای نمونه، ویندوز از کتابخانه‌های اتصال پویا پشتیبانی[1] (DLL) می‌نماید. نفع این رویکرد آن است که از اتصال و بارگذاری کتابخانه‌هایی که ممکن است با فایل اجرایی مورد استفاده قرار نگیرند، اجتناب می‌کند. در عوض، کتابخانه به‌صورت مشروط متصل شده و اگر در طی زمان اجرای برنامه لازم شوند بارگذاری می‌شوند. برای مثال، در شکل ۲-۱۱، کتابخانه‌ی math به فایل اجرایی main متصل نمی‌شوند. به‌جای آن، اتصال‌دهنده اطلاعات مربوط به تجدید مکان را درج می‌کند که به آن امکان می‌دهد تا همچنان‌که برنامه بارگذاری می‌شود به‌صورت پویا متصل شده و بارگذاری شود. در فصل ۹ خواهیم دید که این امکان وجود دارد که پردازه‌های متعددی، کتابخانه‌های اتصالی پویا را به‌اشتراک بگیرند که نتیجه‌اش، صرفه‌جویی قابل توجه در استفاده از حافظه است.

فایل‌های مقصد و فایل‌های اجرایی نوعاً قالب‌های استانداردی دارند که شامل کد ماشین کامپایل شده و یک جدول نماد است که ابرداده‌های مربوط به توابع و متغیرهایی که در برنامه ارجاع داده می‌شوند را در بر می‌گیرد. این قالب استاندارد در سیستم‌های یونیکس و لینوکس به ELF (که اختصار **قالب اجرایی و اتصال‌پذیر**[2] می‌باشد) معروف است. قالب‌های ELF جداگانه‌ای برای فایل‌های با قابلیت تجدید مکان و اجرایی وجود دارند. یک جزء از اطلاعات در فایل ELF فایل‌های اجرایی *نقطه‌ی ورود* برنامه است که شامل آدرس اولین دستورالعملی است که موقع اجرای برنامه اجرا می‌شود. سیستم‌های ویندوز از قالب **اجرایی قابل حمل**[3] (PE) و macOS از قالب Mach-O استفاده می‌کنند.

قالب ELF

لینوکس، فرمان‌های مختلفی را برای شناسایی و ارزیابی فایل‌های ELF فراهم می‌کند. برای مثال، فرمان file نوع یک فایل را معین می‌کند. اگر main.o یک فایل مقصد، و main یک فایل اجرایی باشد، فرمان

```
file main.o
```

گزارش خواهد داد که main.o یک فایل با قابلیت تجدید مکان ELF است، در حالی که فرمان

```
file main
```

گزارش خواهد داد که main یک فایل اجرایی ELF است. فایل‌های ELF به تعدادی بخش تقسیم شده و می‌توانند با استفاده از فرمان readelf ارزیابی می‌شوند.

۲-۶ چرا برنامه‌ها، خاص سیستم‌عامل هستند

اساساً، برنامه‌هایی که روی یک سیستم‌عامل کامپایل می‌شوند بر روی سیستم‌عامل‌های دیگر قابل اجرا نیستند. اگر این‌گونه بود، دنیا جای بهتری بود، و انتخاب ما بر این که از کدام

1. Dynamically linked libraries (DLLs)
2. Executable and Linkable Format (ELF)
3. Portable Executable (PE)

سیستم‌عامل استفاده کنیم به برنامه‌های سودمند و امکانات بستگی داشت به‌جای این که کدام برنامه‌ها در دسترس هستند.

بر اساس بحثی که پیشتر داشتیم، اکنون می‌توانیم بخشی از مسئله را ببینیم—هر سیستم‌عامل یک مجموعه‌ای از فراخوانی‌های سیستمی یکتایی را فراهم می‌کند. فراخوانی‌های سیستمی، بخشی از مجموعه‌ی سرویس‌هایی هستند که سیستم‌عامل‌ها برای استفاده توسط برنامه‌های کاربردی فراهم می‌کنند. حتی اگر فراخوانی‌های سیستمی به‌نحوی یک‌شکل بودند، موانع دیگری، اجرای برنامه‌های کاربردی را بر روی سیستم‌عامل‌های متفاوت برای ما مشکل می‌ساخت. اما اگر شما سیستم‌عامل‌های متعددی را استفاده کرده باشید، ممکن است برخی از برنامه‌های یکسانی را بر روی آن‌ها استفاده کرده باشید. چگونه این امر امکان‌پذیر است؟

یک برنامه می‌تواند با یکی از سه روش زیر برای اجرا بر روی سیستم‌عامل‌های متعدد فراهم شود:

۱- برنامه می‌تواند به یک زبان مفسری نوشته شود (همچون پایتون یا Ruby) که دارای مفسر برای سیستم‌عامل‌های متعددی است. مفسر هر یک از خطوط برنامه‌ی کد اصلی را خوانده، دستورالعمل‌های معادلی را از مجموعه‌ی دستورالعمل‌های بومی اجرا کرده، و فراخوانی‌های بومی سیستم‌عامل را فراخوانی می‌کند. کارایی در مقایسه با برنامه‌های کاربردی بومی متأثر می‌شود، و مفسر تنها یک زیرمجموعه‌ای از امکانات هر سیستم‌عامل را فراهم می‌کند که احتمالاً مجموعه‌ی امکانات برنامه‌های مرتبط را محدود خواهد کرد.

۲- برنامه می‌تواند به یک زبانی نوشته شود که یک ماشین مجازی را include می‌کند و برنامه‌ی اجرایی را در بر می‌گیرد. ماشین مجازی، بخشی از RTE کامل زبان است. یک مثال از این روش، جاوا می‌باشد. جاوا یک RTE دارد که شامل یک بارگذار، وارس[1] بایت‌کد، و سایر مؤلفه‌هاست که برنامه‌ی جاوا را به ماشین مجازی جاوا بارگذاری می‌کنند. این RTE برای بسیاری از سیستم‌عامل‌ها از mainframeها گرفته تا گوشی‌های هوشمند **پورت**[2] یا توسعه داده شده و در تئوری، هر برنامه‌ی جاوا می‌تواند در میان RTE هرجا که باشد اجرا شود. سیستم‌های از این نوع معایبی هم دارند مشابه مواردی که در بالا در مورد مفسرها اشاره شد.

۳- برنامه‌نویس کاربردی می‌تواند از یک زبان یا API استاندارد استفاده کند که در آن، کامپایلر، کدهای دودویی را به یک زبان خاص ماشین و سیستم‌عامل تولید می‌کند. برنامه بایستی در هر سیستم‌عاملی که روی آن اجرا خواهد شد پورت شود. این‌گونه پورت می‌تواند کاملاً زمان‌بر بوده و بایستی برای هر نسخه از برنامه با آزمون و عیب‌یابی بعدی انجام شود. شاید بهترین مثال آشنا، POSIX API و مجموعه‌ی استانداردهای آن برای حفظ سازگاری کد اصلی در میان گونه‌های متفاوت سیستم‌عامل‌های مثل یونیکس باشد.

1. Verifier
2. Port

این سه رویکرد در تئوری به‌نظر می‌رسد که راه‌حل‌های ساده‌ای برای نوشتن برنامه‌هایی که بتوانند در میان سیستم‌عامل‌های متفاوت اجرا شوند فراهم می‌کنند. با این وجود، نداشتن قابلیت حمل کلی برنامه‌ها عوامل متعددی دارد که همگی، نوشتن برنامه‌ها را در میان سکوها با یک چالشی مواجه می‌سازند. در سطح کاربرد، کتابخانه‌های فراهم شده با سیستم‌عامل، شامل APIهایی برای فراهم‌ساختن امکاناتی مثل رابط‌های GUI، و یک برنامه‌ای که طراحی می‌شود تا یک مجموعه از APIها را (برفرض، آن‌هایی که از iOS بر روی iPhone اَپل در دسترس هستند) فراخوانی کند، بر روی سیستم‌عاملی که آن APIها را فراهم نمی‌کند (همچون اندروید) کار نخواهند کرد. چالش‌های دیگری در سطوح پایین‌تر سیستم، اعم از موارد زیر وجود دارند.

- هر سیستم‌عامل برای برنامه‌هایی که چیدمان سرآیند[1]، دستورالعمل‌ها، و متغیرها را دیکته می‌کند یک قالب دودویی دارد. این مؤلفه‌ها نیاز است که در مکان‌های خاصی با ساختارهای مشخصی در میان یک فایل اجرایی باشند تا این که سیستم‌عامل بتواند فایل را باز کرده و برنامه را برای اجرای درست بارگذاری کند.
- پردازنده‌ها دارای مجموعه‌ی دستورالعمل‌های متفاوتی هستند و تنها برنامه‌هایی که مشمول دستورالعمل‌های مناسبی هستند می‌توانند به‌درستی اجرا شوند.
- سیستم‌عامل‌ها فراخوانی‌های سیستمی فراهم می‌کنند که به برنامه‌ها اجازه می‌دهد تا فعالیت‌های مختلفی را درخواست بدهند، همچون ایجاد فایل‌ها و باز کردن اتصالات شبکه. این فراخوانی‌های سیستمی در میان سیستم‌عامل‌ها در وجوه بسیاری با هم فرق دارند، اعم از عملوندهای خاص و ترتیب عملوند استفاده شده، چگونگی احضار فراخوانی‌های سیستمی توسط برنامه‌ی کاربردی، شماره‌گذاری و شماره، معانی آن‌ها، و نتایج برگشتی آن‌ها.

برخی رویکردهایی هستند که برای رفع این اختلاف‌های معماری کمک می‌کنند، هرچند که راه‌حل کاملی نیستند. برای مثال، لینوکس—و تقریباً هر سیستم یونیکسی—قالب ELF را برای فایل‌های اجرایی دودویی تطبیق داده‌اند. هرچند که ELF یک استاندارد مشترکی در میان سیستم‌های لینوکس و یونیکس فراهم می‌کند، قالب ELF به هر معماری کامپیوتری خاصی گره نمی‌خورد، لذا تضمین نمی‌کند که یک فایل اجرایی در میان سکوهای سخت‌افزاری متفاوت اجرا خواهد شد.

APIها، همان‌گونه که در بالا ذکر شد، کارکردهای خاصی را در سطح کاربرد توصیف می‌کنند. در سطح معماری، یک **رابط دودویی کاربرد**[2] (**ABI**) استفاده می‌شود تا چگونگی ارتباط مؤلفه‌های متفاوت کد دودویی یک سیستم‌عامل مشخص بر روی یک معماری مشخص را تعریف کند. یک ABI جزئیات سطح پایین را توصیف می‌کند، اعم از پهنای آدرس، روش‌های ارسال پارامترها به فراخوانی‌های سیستمی، سازمان پشته‌ی زمان اجرا، قالب دودویی کتابخانه‌های سیستمی، و اندازه‌ی انواع داده‌ها که تنها نمونه‌هایی از موارد مد نظر می‌باشند. یک ABI نوعاً برای

1. Header
2. Application binary interface (ABI)

معماری مشخصی توصیف می‌شود (برای مثال، یک ABI برای پردازنده‌ی ARMv8 وجود دارد). بنابراین، یک ABI، معادل سطح معماری یک API است. اگر یک فایل اجرایی دودویی کامپایل شده باشد و مطابق یک ABI خاصی متصل شود، بایستی قادر باشد بر روی سیستم‌های متفاوتی که آن ABI را پشتیبانی می‌کنند اجرا شود. با این همه، چون یک ABI به‌خصوص برای سیستم‌عامل خاصی که روی یک معماری مشخصی اجرا می‌شود تعریف می‌شود، ABIها در تدارک سازگاری بین سکویی، کار کمی از پیش می‌برند.

در مجموع، تمامی این اختلاف‌ها به این معنی است که مگر این که یک مفسر، RTE، یا یک فایل اجرایی دودویی نوشته شده و روی یک سیستم‌عامل خاصی بر روی یک نوع پردازنده‌ی خاصی (همچون اینتل x86 یا ARMv8) کامپایل شود، برنامه در اجرا با شکست مواجه خواهد شد. میزان کاری را که برای یک برنامه همچون مرورگر فایرفاکس لازم است تصور کنید تا بر روی ویندوز، macOS، نسخه‌های مختلف لینوکس، iOS، و اندروید، بعضاً هم بر روی معماری‌های مختلف CPU اجرا شود.

۲-۷ طراحی و پیاده‌سازی سیستم‌عامل

در این بخش، مسائلی را بحث می‌کنیم که در طراحی و پیاده‌سازی یک سیستم‌عامل مواجه می‌شویم. البته، راه‌حل کاملی برای این قبیل مسائل وجود ندارد، اما رویکردهایی هستند که موفقیت آنها به اثبات رسیده است.

۲-۷-۱ اهداف طراحی

اولین مسأله در طراحی یک سیستم، تعریف و توصیف اهداف آن است. در بالاترین سطح، طراحی سیستم، تحت تأثیر گزینش سخت‌افزار و نوع سیستم خواهد بود: سیستم رومیزی سابق/لپ‌تاپ، موبایل، توزیع‌شده، یا بلادرنگ.

توصیف نیازمندی‌ها که بالاتر از این سطح طراحی هم قرار دارد می‌تواند بسیار دشوارتر باشد. با این وجود، نیازمندی‌ها می‌توانند به دو گروه اساسی تقسیم شوند که عبارتند از: **اهداف کاربر** و **اهداف سیستم**.

کاربران خصوصیات روشن خاصی از یک سیستم را می‌خواهند. سیستم بایستی کاربرد راحت، یادگیری و استفاده‌ی آسانی داشته، مطمئن، ایمن و سریع باشد. البته، این توصیف‌ها مخصوصاً در طراحی سیستم سودمند نیستند، چرا که هیچ‌گونه توافق کلی برای حصول این اهداف وجود ندارد.

مجموعه‌ی مشابهی از نیازمندی‌ها هم می‌تواند توسط توسعه‌دهندگانی که می‌بایست سیستم را طراحی، ایجاد، نگه‌داری و عملیاتی کنند، تعریف شود. سیستم بایستی طراحی، پیاده‌سازی و نگه‌داری آسانی داشته و انعطاف‌پذیر، مطمئن، عاری از خطا و کارآمد باشد. باز هم این نیازمندی‌ها مبهم هستند و به شیوه‌های مختلفی قابل تفسیرند.

به طور خلاصه، هیچ راه‌حل واحدی برای مسأله‌ی تعریف نیازمندی‌های یک سیستم‌عامل وجود ندارد. گستردگی سیستم‌های موجود نشان می‌دهد که نیازمندی‌های متفاوت می‌توانند تنوع